Weather App

By Ryan Rupakheti

Overview of Functionality

What my Weather App Does

- Display current weather
- 5-Day Forecast
- Save favorite cities
- Temperature Conversion

What I used or implement:

- Tkinter
- Openweather API
- ThreadPoolExecutor
- Pillow
- Datetime
- Exception Handling for files and user input

Getting API Data

- Had to create two API urls and get City Input.
- For forecast dictionaries had a list with 40 dictionaries that record data every 3 hours.
- Used ThreadPool to get API data quicker.

```
with ThreadPoolExecutor() as executor:
    CDweather = executor.submit(Get_Weather_Data,self.City_Input,self.City_Units)
    CFweather = executor.submit(Get_ForeCast,self.City_Input,self.City_Units)
```

```
#Here we get Basic Daily ForCast Information
City_Temp = round(Current_City['main']['temp'])
City_Feels_Temp = round(Current_City['main']['feels_like'])
City_Max = round(Current_City['main']['temp_max'])
City_Min = round(Current_City['main']['temp_min'])
City_Humidity = Current_City['main']['humidity']
City_Description = Current_City['weather'][0]['description'].upper()
City_Wind = Current_City['wind']['speed']
City_Icon = Current_City['weather'][0]['icon']
```

```
#Stores only needed data
five_day = {}

# For each forecast in Five_Day_F
for forecast in Five_Day_F['list']:
    # Get the date
    date = forecast['dt_txt'].split()[0]
    #If not in five_days intilize the data
    if date not in five_day:
        five_day[date]={
            'high': round(forecast['main']['temp_max']),
            'low': round(forecast['main']['temp_min']),
            'icon': forecast['weather'][0]['icon']
        }
    #Else comp current value with the current max and forecasr and update
    else:
        five_day[date]['high'] = round(max(five_day[date]['high'], forecast['main']['temp_max']))
        five_day[date]['low'] = round(min(five_day[date]['low'], forecast['main']['temp_min']))


for key,value in five_day.items():
    Cdate = dt.strptime(key,"%Y-%m-%d")
    weekday = Cdate.weekday()
    five_day[key]["day"] = cd.day_abbr[weekday]
```

Save Cities

Another important thing is to save certain cities.

To do this I used a list to store the inputs of cities saved.

Then write the list onto a text file, so buttons are saved when closed.

Read it when homepage opens for buttons.

```
self.SavedCity = []
try:
    with open("CityButtons.txt","r") as f:
        self.SavedCity = f.readline()
        self.SavedCity = self.SavedCity.split(",")
except FileNotFoundError:
    messagebox.showerror("No File",'File not found')


self.CityButton = []
```

```
293     def Save_City(self):
294
295         if self.City_Input not in self.SavedCity:
296             self.SavedCity.append(self.City_Input)
297
298         if self.City_Input in self.SavedCity:
299             self.SaveB.config(text="Remove",command=self.Remove_City)
300
301         self.UpdateFile()
302
303
304
305     def Remove_City(self):
306
307         if self.City_Input in self.SavedCity:
308             self.SavedCity.remove(self.City_Input)
309             self.SaveB.config(text="Save",command=self.Save_City)
310
311         self.UpdateFile()
312
313
314     def UpdateFile(self):
315         try:
316             with open("CityButtons.txt","w") as f:
317                 f.write(",".join(self.SavedCity))
318         except FileNotFoundError:
319             messagebox.showerror("No File","File not Found")
320             return
```

Output on GUI

Main Output:

- This was handled by three functions
- Display_Info is where we get text input and display output
- Update_Labels displays and updates the UI for current forecast
- Display_Forecast displays and updates the UI for 5 forecast

Helper Functions:

- Convert function that will set the metric for api calls
- Save and Remove that stores a list
- Function to help update the file
- Finally, one that updates homepage

Improvement

- I did not spend much time on GUI design
- This app is still slow so better use of threading or maybe async.
- Reduce redundant code